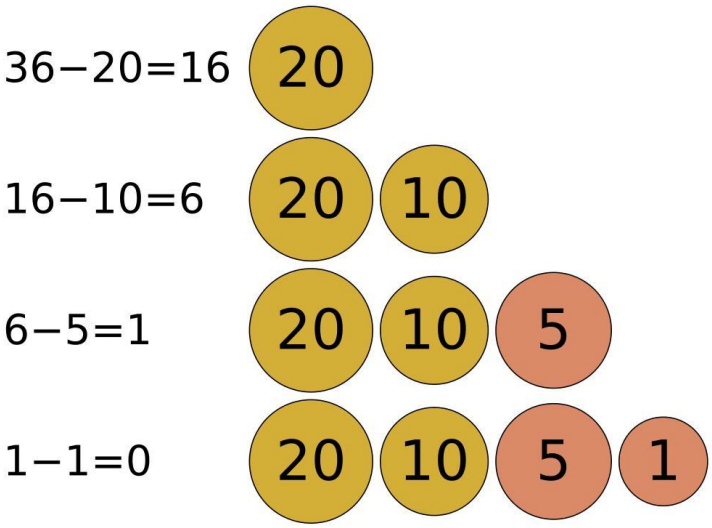
Algoritmos Gulosos vs. Programação Dinâmica

Os dois são estratégias de projeto de algoritmos usadas para resolver **problemas de otimização**, isto é, problemas em que queremos encontrar a **melhor solução possível**, como o menor custo, o maior lucro ou o menor número de operações.

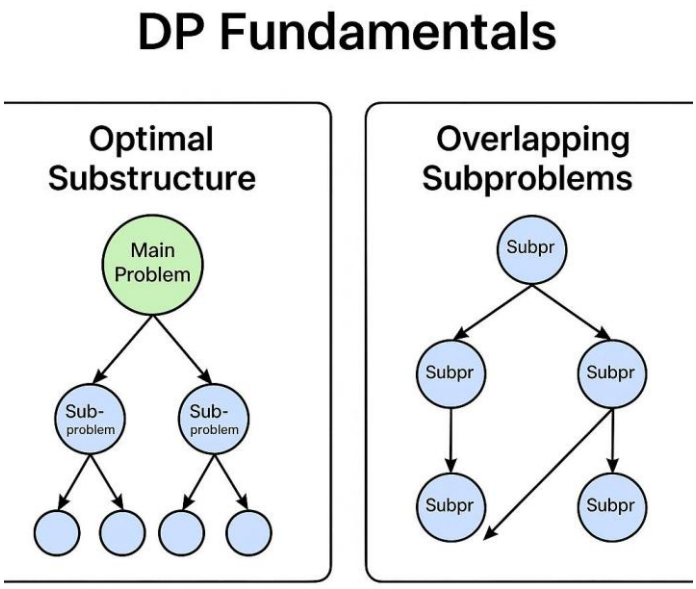
A principal diferença está em **como o algoritmo toma decisões** para chegar à solução.



Algoritmo Guloso (Greedy)

Escolhe a melhor opção disponível naquele momento, sem necessariamente avaliar todas as consequências futuras.

Decisão local



Programação Dinâmica (PD)

Resolve subproblemas menores, guarda os resultados e os combina para encontrar uma solução ótima.

Subproblemas

1. Exemplo de algoritmo guloso: sistema de troco

Imagine que precisamos devolver **R$ 36** utilizando moedas de **R$ 25, R$ 10, R$ 5 e R$** 1, buscando utilizar o menor número possível de moedas.

O algoritmo guloso escolhe, a cada passo, a maior moeda que não ultrapassa o valor restante.

Execução passo a passo

R$ 25

1ª escolha

36 − 25 = 11 reais restantes

R$ 10

2ª escolha

11 − 10 = 1 real restante

R$ 1

3ª escolha

1 − 1 = 0. Troco concluído!

Resultado: 3 moedas

R$ 25 + R$ 10 + R$ 1 = R$ 36

Como seria o algoritmo?

```
def troco_guloso(valor, moedas):
    moedas_usadas = []
    for moeda in sorted(moedas, reverse=True):
        while valor >= moeda:
            valor -= moeda
            moedas_usadas.append(moeda)
    return moedas_usadas
print(troco_guloso(36, [25, 10, 5, 1]))
```

Resultado: [25, 10, 1]

A cada etapa, o algoritmo toma a melhor decisão imediata: usar a maior moeda possível.

Atenção: a estratégia gulosa não encontra necessariamente a solução ótima em qualquer sistema de moedas. Ela funciona para esse exemplo, mas pode falhar em outros sistemas.

2. Exemplo de programação dinâmica: sistema de troco

Agora, vamos utilizar moedas de 1, 3 e 4 reais para devolver R$ 6, novamente buscando o menor número de moedas.

Um algoritmo guloso escolheria primeiro a moeda de R$ 4:

- R$ 6 − R$ 4 = R$ 2.
- Depois, escolheria duas moedas de R$ 1.

Resultado guloso: **3 moedas** — R$ 4 + R$ 1 + R$ 1.

Mas existe uma **solução melhor:** 6 = 3 + 3

Resultado ótimo: **2 moedas**.

A **programação dinâmica** encontra essa solução comparando as alternativas para os valores menores.

Como ela calcula?

Definimos **DP[x]** como o número mínimo de moedas necessárias para formar o valor x.

A fórmula de recorrência é:

$$DP[x] = 1 + \min_{m \le x} DP[x - m]$$

Aqui, m representa cada moeda disponível: 1, 3 ou 4 reais. Inicialmente, DP[0] = 0.

Valor \(x\)	*Menor número de moedas*	*Uma solução ótima*
0	0	Nenhuma moeda
1	1	1
2	2	1 + 1
3	1	3
4	1	4
5	2	4 + 1
6	2	3 + 3

O algoritmo considera as alternativas e guarda o melhor resultado de cada valor calculado.

Código em Python

```
def troco_dinamico(valor, moedas):
    dp = [float('inf')] * (valor + 1)
    dp[0] = 0
    for x in range(1, valor + 1):
        for moeda in moedas:
            if moeda <= x:
                dp[x] = min(
                    dp[x],
                    dp[x - moeda] + 1
                )
    return dp[valor]
print(troco_dinamico(6, [1, 3, 4]))
```

Resultado: 2

O algoritmo retorna o número mínimo de moedas. Para também apresentar quais moedas foram usadas, podemos guardar a escolha que produziu cada melhor resultado.

3. Comparação direta

Característica	*Guloso*	*Programação dinâmica*
Estratégia	Escolhe a melhor opção imediata	Compara alternativas por meio de subproblemas
Decisões anteriores	Geralmente não são revistas	Resultados são reutilizados
Armazenamento	Frequentemente menor	Usa memória para guardar resultados
Desempenho	Frequentemente mais rápido e simples	Pode exigir mais tempo e memória
Garante solução ótima?	Somente quando a estratégia gulosa é válida para o problema	Quando a recorrência e os subproblemas são formulados corretamente
Exemplo	Troco em sistemas apropriados	Troco com moedas de 1, 3 e 4

4. Como diferenciar em uma questão de prova?

Pense nestas duas perguntas:

- **Guloso**: "Qual é a **melhor escolha** que posso fazer agora?"
- **Programação dinâmica**: "Qual é a **melhor solução** que consigo construir considerando os resultados dos subproblemas?"

Um detalhe importante: **programação dinâmica** não significa necessariamente testar todas as soluções completas. Ela evita repetir cálculos ao armazenar resultados de **subproblemas**, usando técnicas como **memoização** ou uma tabela de resultados (**tabulação**).

Para **memorizar**: o algoritmo guloso escolhe localmente; a **programação dinâmica** combina **soluções ótimas** de subproblemas para resolver o **problema maior**.